

**Học Viện Công Nghệ Bưu Chính Viễn Thông**

**Khoa Công nghệ Thông Tin 1**

====o0o====



**BÁO CÁO CUỐI KỲ**  
**MÔN IOT VÀ ỨNG DỤNG**

**ĐỀ TÀI: XÂY DỰNG HỆ THỐNG IOT**  
**NHÀ THÔNG MINH**

Giảng viên hướng dẫn: TS. Kim Ngọc Bách

Nhóm lớp 06 – nhóm 01

| Mã sinh viên | Họ và tên         |
|--------------|-------------------|
| B22DCCN313   | Nguyễn Duy Hiếu   |
| B22DCCN122   | Bùi Tiến Dũng     |
| B22DCCN133   | Nguyễn Quang Dũng |
| B22DCCN182   | Đào Mạnh Đạt      |

## Mục Lục

|                                                                 |    |
|-----------------------------------------------------------------|----|
| A. Giới thiệu đề tài .....                                      | 4  |
| B. Công nghệ và lý thuyết áp dụng .....                         | 5  |
| I. Cơ sở lý thuyết .....                                        | 5  |
| II. Các công nghệ sử dụng .....                                 | 5  |
| III. Các lý thuyết nền tảng áp dụng .....                       | 7  |
| IV. Mối liên hệ giữa công nghệ và các Use Case .....            | 7  |
| C. Phân tích yêu cầu - Các tính năng triển khai .....           | 9  |
| I. Xác định mục tiêu & phạm vi hệ thống .....                   | 9  |
| 1. Mục tiêu .....                                               | 9  |
| 2. Phạm vi triển khai .....                                     | 9  |
| 3. Tiêu chí thành công (dự kiến) .....                          | 9  |
| 4. Kết quả mong đợi .....                                       | 10 |
| II. Thu thập yêu cầu từ stakeholders .....                      | 10 |
| 1. Người dùng hệ thống (chủ hộ) .....                           | 10 |
| 2. Kỹ thuật/IT .....                                            | 10 |
| III. Xác định yêu cầu chức năng .....                           | 10 |
| 1. Các chức năng cần có của hệ thống (dự kiến triển khai) ..... | 10 |
| 2. Đặc tả luồng công việc (use case, sequence diagram) .....    | 11 |
| IV. Xác định yêu cầu phi chức năng .....                        | 12 |
| 1. Hiệu năng : .....                                            | 12 |
| 2. Tính bảo mật (Security) .....                                | 12 |
| 3. Tính ổn định và tin cậy (Reliability & Availability) .....   | 13 |
| 4. Khả năng mở rộng (Scalability & Extensibility) .....         | 13 |
| 5. Tính thân thiện và dễ sử dụng (Usability) .....              | 13 |
| V. Phân tích ràng buộc .....                                    | 14 |
| VI. Mô hình hóa yêu cầu .....                                   | 14 |

|                                                                              |    |
|------------------------------------------------------------------------------|----|
| 1. Mô hình tổng thể hệ thống.....                                            | 14 |
| 2. Biểu đồ Use Case tổng quát.....                                           | 14 |
| 3. Biểu đồ luồng dữ liệu (Data Flow Diagram).....                            | 15 |
| 4. Mô hình kiến trúc triển khai .....                                        | 16 |
| D. Phân tích thiết kế .....                                                  | 16 |
| I. Biểu đồ tuần tự (Sequence Diagram) .....                                  | 16 |
| II. Biểu đồ lớp (Class Diagram) .....                                        | 19 |
| III. Thiết kế MVC .....                                                      | 20 |
| 1. Thiết kế chi tiết tầng View.....                                          | 23 |
| 2. Thiết kế chi tiết tầng Controller.....                                    | 26 |
| IV. Thiết kế cơ sở dữ liệu .....                                             | 28 |
| E. KẾT QUẢ ĐẠT ĐƯỢC .....                                                    | 32 |
| 1. Giao diện web điều khiển.....                                             | 32 |
| 2. Điều khiển thiết bị từ xa .....                                           | 32 |
| 3. Cảm biến gửi dữ liệu lên web liên tục và gửi cảnh báo khi vượt ngưỡng ... | 33 |
| 4. Hiện thị lịch sử người dùng .....                                         | 34 |
| 5. Nhật ký nhiệt độ, độ ẩm và ánh sáng.....                                  | 34 |
| F. KẾT LUẬN & HƯỚNG MỞ RỘNG .....                                            | 35 |
| I. Kết luận .....                                                            | 35 |
| II. Hướng mở rộng.....                                                       | 36 |
| 1. Bổ sung thêm cảm biến và thiết bị .....                                   | 36 |
| 2. Phát triển cơ chế cảnh báo từ xa.....                                     | 36 |
| 3. Nâng cấp bảo mật .....                                                    | 36 |
| 4. Tối ưu OTA và quản lý firmware .....                                      | 36 |
| 5. Hướng tới triển khai thực tế .....                                        | 37 |
| G. Phân chia công việc và kế hoạch triển khai .....                          | 38 |

## A. Giới thiệu đề tài

### Lý do chọn đề tài

Trong bối cảnh cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, nhà thông minh (Smart Home) trở thành một trong những xu hướng công nghệ nổi bật, minh chứng cho sự phát triển vượt bậc của Internet vạn vật (IoT – Internet of Things). Việc lựa chọn nghiên cứu đề tài này xuất phát từ nhu cầu thực tế về sự tiện nghi, an toàn và tối ưu hóa năng lượng trong sinh hoạt hằng ngày.

Con người hiện đại ngày càng chú trọng đến việc nâng cao chất lượng cuộc sống thông qua những giải pháp công nghệ giúp tự động hóa, tiết kiệm thời gian và tăng cường bảo mật. Hệ thống nhà thông minh không chỉ đáp ứng những nhu cầu đó mà còn thể hiện xu hướng tất yếu của thời đại số.

Bên cạnh đó, đề tài mở ra cơ hội tìm hiểu, nghiên cứu và ứng dụng các công nghệ tiên tiến như cảm biến, vi điều khiển, giao thức truyền thông (MQTT, Zigbee, Bluetooth), cũng như trí tuệ nhân tạo (AI) trong việc xử lý và điều khiển thiết bị. Đây là nền tảng quan trọng để sinh viên tiếp cận thực tế, gắn kết lý thuyết với ứng dụng.

Đề tài còn mang ý nghĩa xã hội sâu sắc khi góp phần tiết kiệm năng lượng, giảm thiểu rủi ro cháy nổ và bảo vệ môi trường thông qua việc sử dụng công nghệ hiệu quả. Hơn nữa, mô hình nhà thông minh có thể được ứng dụng rộng rãi trong nhiều lĩnh vực như gia đình, doanh nghiệp, giáo dục và y tế. Do đó, việc nghiên cứu và triển khai đề tài không chỉ có giá trị học thuật mà còn mang tính thực tiễn và định hướng phát triển công nghệ trong tương lai.

“Vì vậy, nhóm em quyết định lựa chọn đề tài *‘Phân tích và thiết kế hệ thống Nhà thông minh ứng dụng IoT’* nhằm xây dựng mô hình tự động hóa các thiết bị trong gia đình, hướng đến cuộc sống hiện đại, tiện nghi và an toàn hơn.”

## B. Công nghệ và lý thuyết áp dụng

### I. Cơ sở lý thuyết

Hệ thống nhà thông minh (Smart Home) là một trong những ứng dụng tiêu biểu của Internet of Things (IoT) – công nghệ cho phép các thiết bị vật lý có khả năng thu thập, trao đổi và xử lý dữ liệu thông qua mạng Internet. Mô hình hoạt động của hệ thống được chia thành bốn lớp cơ bản:

- ❖ *Lớp cảm nhận (Perception Layer)*: Bao gồm các cảm biến như DHT22, LDR có nhiệm vụ thu thập thông tin môi trường (nhiệt độ, độ ẩm, ánh sáng).
- ❖ *Lớp mạng (Network Layer)*: Truyền dữ liệu cảm biến qua WiFi bằng vi điều khiển ESP32, sử dụng các giao thức truyền thông nhẹ như MQTT hoặc HTTP/HTTPS.
- ❖ *Lớp xử lý (Processing Layer)*: Server backend tiếp nhận, xử lý dữ liệu, đưa ra quyết định tự động (ví dụ bật đèn khi trời tối).
- ❖ *Lớp ứng dụng (Application Layer)*: Giao diện người dùng trên web hoặc ứng dụng di động giúp theo dõi thông số thời gian thực và điều khiển thiết bị từ xa.

Nhờ sự phối hợp giữa các lớp này, hệ thống có thể giám sát liên tục và phản hồi nhanh chóng, tạo nên một môi trường sinh hoạt hiện đại, tiện nghi và an toàn.

### II. Các công nghệ sử dụng

#### 1. Vi điều khiển và cảm biến

- *ESP32*: Là bộ vi điều khiển chính của hệ thống, tích hợp WiFi và Bluetooth, chịu trách nhiệm thu thập dữ liệu từ cảm biến và truyền lên server.
- *Cảm biến DHT22*: Đo nhiệt độ và độ ẩm môi trường, phục vụ các chức năng tự động bật quạt khi nhiệt độ cao hoặc cảnh báo khi điều kiện vượt ngưỡng.

- *Cảm biến LDR*: Đo cường độ ánh sáng, giúp hệ thống tự động bật đèn khi trời tối.
- *Relay module*: Relay dùng để bật/tắt các thiết bị điện (đèn, quạt).

## 2. Giao thức truyền thông

Hệ thống sử dụng kết hợp hai giao thức phổ biến trong IoT:

- *MQTT (Message Queuing Telemetry Transport)*:  
Là giao thức truyền thông dạng *publish/subscribe*, cho phép các thiết bị (ESP32) gửi dữ liệu lên broker (ví dụ HiveMQ Cloud) và các ứng dụng đăng ký nhận dữ liệu. Ưu điểm của MQTT là tốc độ nhanh, dung lượng gói tin nhỏ, độ trễ thấp (<800ms), phù hợp với truyền dữ liệu cảm biến thời gian thực.
- *HTTP/HTTPS (REST API)*:  
Được sử dụng trong việc quản lý, xác thực người dùng và lấy dữ liệu lịch sử. HTTPS (TLS 1.2+) giúp mã hóa đường truyền, đảm bảo an toàn khi giao tiếp giữa ứng dụng và server.

## 3. Backend và cơ sở dữ liệu

- *Server Backend*: Được phát triển theo mô hình RESTful API, có nhiệm vụ kết nối và quản lý các thiết bị IoT, xử lý logic điều khiển, đồng thời giao tiếp với MQTT broker để nhận hoặc gửi dữ liệu.
- *Cơ sở dữ liệu (MySQL)*: Lưu trữ dữ liệu cảm biến, lịch sử điều khiển, và thông tin người dùng. Dữ liệu được ghi nhận theo thời gian thực giúp người dùng xem lại và phân tích tình trạng môi trường.

## 4. Ứng dụng điều khiển (Web/App)

Ứng dụng được phát triển trên nền tảng MIT App Inventor, giao diện thân thiện, hiển thị thông số môi trường và trạng thái thiết bị theo thời gian thực. Người dùng có thể:

- Bật/tắt đèn, quạt qua nút điều khiển hoặc lệnh giọng nói.

- Theo dõi giá trị nhiệt độ, độ ẩm, ánh sáng.
- Nhận thông báo cảnh báo khi có sự cố bất thường.
- Đăng nhập và phân quyền truy cập (admin hoặc người dùng thường).

### III. Các lý thuyết nền tảng áp dụng

| Lý thuyết                                   | Ứng dụng trong hệ thống                                                                                                     |
|---------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Internet of Things (IoT)                    | Là nền tảng chính giúp các thiết bị cảm biến và điều khiển giao tiếp với nhau qua mạng Internet.                            |
| Truyền thông MQTT/HTTP                      | Dùng để truyền và nhận dữ liệu giữa ESP32, server và ứng dụng.                                                              |
| Cảm biến học (Sensor Theory)                | Giúp đo và xử lý tín hiệu vật lý (ánh sáng, nhiệt độ, độ ẩm).                                                               |
| Điều khiển tự động (Automation Logic)       | Áp dụng các điều kiện ngưỡng (ví dụ nhiệt độ $>30^{\circ}\text{C}$ $\rightarrow$ bật quạt; trời tối $\rightarrow$ bật đèn). |
| Lý thuyết an toàn mạng và bảo mật IoT       | Sử dụng mã hóa TLS, xác thực bằng token JWT, đảm bảo dữ liệu và quyền truy cập an toàn.                                     |
| Phân tích và thiết kế hướng đối tượng (UML) | Dùng để mô tả các Use Case (UC1–UC9), luồng xử lý và hoạt động của hệ thống.                                                |
| Phát triển phần mềm hướng dịch vụ (SOA)     | Giúp tách biệt rõ ràng giữa ba phần: thiết bị IoT, server backend và ứng dụng người dùng.                                   |

### IV. Mối liên hệ giữa công nghệ và các Use Case

| Use Case                     | Công nghệ & lý thuyết áp dụng                       |
|------------------------------|-----------------------------------------------------|
| UC1 – UC2: Bật/Tắt đèn, quạt | ESP32 + Relay module + MQTT + Điều khiển giọng nói. |

|                                             |                                                      |
|---------------------------------------------|------------------------------------------------------|
| UC3 – UC4: Đo nhiệt độ, độ ẩm               | Cảm biến DHT22 + xử lý tín hiệu + MQTT publish.      |
| UC5: Tự động bật đèn khi trời tối           | Cảm biến LDR + Logic điều khiển tự động.             |
| UC6: Gửi dữ liệu cảm biến lên server        | MQTT hoặc HTTP + mô hình truyền dữ liệu IoT bốn lớp. |
| UC7 – UC8: Cảnh báo ngưỡng nhiệt độ & độ ẩm | Xử lý cảnh báo + gửi thông báo qua Web/App.          |
| UC9: Lưu lịch sử dữ liệu                    | RESTful API + Database (MySQL).                      |

## C. Phân tích yêu cầu - Các tính năng triển khai

### I. Xác định mục tiêu & phạm vi hệ thống

#### 1. Mục tiêu

##### a. Mục tiêu tổng quát:

- Nâng cao tiện nghi, tiết kiệm năng lượng, và đảm bảo an toàn cho người sử dụng.
- Xây dựng mô hình minh chứng ứng dụng công nghệ IoT trong đời sống thực tế.

##### b. Mục tiêu cụ thể:

- Xây dựng hệ thống nhà thông minh cho phép giám sát và điều khiển các thiết bị điện trong nhà (đèn, quạt, cảm biến môi trường: nhiệt độ, độ ẩm, ánh sáng, v.v.) từ xa qua Internet.
- Cung cấp giao diện web hoặc mobile app giúp người dùng dễ dàng thao tác và theo dõi dữ liệu thời gian thực.
- Ứng dụng điều khiển bằng giọng nói và cảnh báo chuyển động bất thường.

##### c. Mục tiêu IoT cụ thể:

- Giám sát tự động các chỉ số môi trường: nhiệt độ, độ ẩm, ánh sáng.
- Các chế độ tự động: bật/tắt quạt, đèn theo nhiệt độ – ánh sáng theo tình huống.
- Cảnh báo khi xuất hiện tiếng động lạ.

#### 2. Phạm vi triển khai

- **Phần cứng:**  
Bao gồm vi điều khiển **ESP32**, cảm biến **DHT22 (nhiệt độ – độ ẩm)**, **LDR (ánh sáng)**, cùng **relay module 5V** và **servo** để điều khiển thiết bị.
- **Phần mềm:**  
Ứng dụng điều khiển (web/app), cơ sở dữ liệu lưu trữ, và hệ thống truyền thông (MQTT/HTTP).
- **Người dùng:**  
Chủ hộ, các thành viên gia đình, và quản trị hệ thống.

#### 3. Tiêu chí thành công (dự kiến)

- Độ chính xác: Sai số đo độ ẩm  $< \pm 5\%$ .

- Độ trễ: Dữ liệu cảm biến cập nhật lên server trong vòng  $< 5$  giây.
- Độ tin cậy: Tỷ lệ truyền dữ liệu thành công  $> 98\%$ .
- Khả năng mở rộng: Hệ thống có thể số lượng cảm biến trong thời gian tới mà không cần thay đổi hạ tầng mạng.

#### **4. Kết quả mong đợi**

- Tự động hóa các công việc cơ bản  $\rightarrow$  tiết kiệm thời gian.
- Người chủ hộ và các thành viên giám sát từ xa qua smartphone.
- Dữ liệu thu thập liên tục giúp đưa ra quyết định tốt hơn (ví dụ: tự động bật đèn khi trời tối).

## **II. Thu thập yêu cầu từ stakeholders**

### **1. Người dùng hệ thống (chủ hộ)**

- Hiển thị: muốn thấy tình trạng ngôi nhà (nhiệt độ, độ ẩm, ánh sáng) ngay trên màn hình dashboard hoặc trên app di động.
- Cảnh báo: nhận thông báo tức thì khi có dấu hiệu bất thường (ví dụ: nhiệt độ, độ ẩm cao ...v.v).
- Điều khiển: điều khiển từ xa các thiết bị đã yêu cầu (đèn, quạt, ...v.v).

### **2. Kỹ thuật/IT**

- Đầy đủ chức năng, đúng quy trình IoT, có tính ứng dụng thực tế
- Có dữ liệu cảm biến rõ ràng, API ổn định, dễ mở rộng

## **III. Xác định yêu cầu chức năng**

### **1. Các chức năng cần có của hệ thống (dự kiến triển khai)**

1. Thu thập và hiển thị giá trị nhiệt độ, độ ẩm, ánh sáng, chuyển động theo thời gian thực
2. Bật/tắt đèn, quạt qua ứng dụng hoặc web
3. Đèn bật khi trời tối, quạt bật khi nhiệt độ cao
4. Điều khiển bằng giọng nói
5. Gửi thông báo (Web/App) khi có sự cố bất thường
6. Hỗ trợ đăng nhập, phân quyền (admin, người dùng thường)
7. Ghi lại lịch sử cảm biến và hoạt động điều khiển
8. Bảng điều khiển hiển thị dữ liệu real-time và trạng thái thiết bị

## 2. Đặc tả luồng công việc (use case, sequence diagram)

### a, Use Case Diagram:

1. Tác nhân: Chủ hộ, Quản trị viên.
2. Các use case chính:

| Use case | Tên                                                | Mô tả                                                                                                                                 | Luồng sự kiện chính                                                                      |
|----------|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| UC1      | Bật/Tắt đèn                                        | Người dùng điều khiển đèn qua ứng dụng hoặc lệnh giọng nói. Hệ thống nhận tín hiệu và gửi lệnh bật/tắt đến module điều khiển (relay). | 1. Người dùng gửi lệnh<br>2. Hệ thống điều khiển relay<br>3. Phản hồi trạng thái lại app |
| UC2      | Bật/Tắt quạt                                       | Tương tự UC1, người dùng điều khiển quạt qua app hoặc ra lệnh bằng giọng nói.                                                         | 1. Người dùng gửi lệnh<br>2. Hệ thống điều khiển relay<br>3. Phản hồi trạng thái lại app |
| UC3      | Đo nhiệt độ                                        | Cảm biến DHT22 đo nhiệt độ môi trường định kỳ (1–2 Hz) và gửi về vi điều khiển.                                                       | IoT thu và xử lý giá trị định kỳ                                                         |
| UC4      | Đo độ ẩm                                           | Đo độ ẩm môi trường và gửi dữ liệu lên server.                                                                                        | IoT thu và xử lý giá trị định kỳ                                                         |
| UC5      | Đo ánh sáng                                        | Đo ánh sáng môi trường rồi gửi dữ liệu lên server                                                                                     | IoT thu và xử lý giá trị định kỳ                                                         |
| UC6      | Gửi dữ liệu nhiệt độ, độ ẩm và ánh sáng lên server | IoT node định kỳ gửi dữ liệu cảm biến lên server qua MQTT hoặc HTTP API.                                                              | IoT gói dữ liệu và gửi MQTT/HTTP                                                         |
| UC7      | Lưu lịch sử dữ liệu cảm biến                       | Server lưu trữ dữ liệu vào cơ sở dữ liệu để phục vụ                                                                                   | Dữ liệu lưu DB theo thời gian                                                            |

|     |                              |                                                                       |                                                                                  |
|-----|------------------------------|-----------------------------------------------------------------------|----------------------------------------------------------------------------------|
|     |                              | thống kê và hiển thị lịch sử.                                         |                                                                                  |
| UC8 | Tự động bật đèn khi trời tối | Cảm biến ánh sáng đo cường độ, nếu dưới ngưỡng → gửi tín hiệu bật đèn | 1. Cảm biến phát hiện ánh sáng thấp<br>2. Gửi tín hiệu đến IoT<br>3. IoT bật đèn |

#### b, Activity Diagram:

Mô tả luồng tự động:

- Cảm biến → Vi điều khiển → Gửi dữ liệu MQTT → Server → Web/App hiển thị.
- Khi vượt ngưỡng → gửi cảnh báo + bật/tắt thiết bị tương ứng.

#### IV. Xác định yêu cầu phi chức năng

##### 1. Hiệu năng :

###### ▪ Độ trễ điều khiển:

- Thời gian từ khi người dùng gửi lệnh đến khi thiết bị phản hồi < 1 giây.
- Lệnh MQTT/HTTP phải được xử lý gần như tức thời (< 800 ms).

###### ▪ Tần suất cập nhật cảm biến

- Cảm biến gửi dữ liệu lên server **mỗi 5 giây/lần** (hoặc có thể điều chỉnh trong cấu hình).

###### ▪ Tốc độ hiển thị dashboard:

- Dashboard/App phải hiển thị dữ liệu real-time và cập nhật trơn tru, không giật lag.

###### ▪ Dung lượng truyền:

- Gói dữ liệu cảm biến nhỏ gọn (< 1 KB/gói) để tiết kiệm băng thông WiFi.

##### 2. Tính bảo mật (Security)

###### ▪ Mã hóa dữ liệu truyền:

- Sử dụng MQTTS hoặc HTTPS (TLS 1.2+) để đảm bảo an toàn đường truyền.

###### ▪ Xác thực & phân quyền:

- Người dùng phải đăng nhập trước khi truy cập hệ thống

- Có phân quyền rõ ràng:
  - ◆ *Admin*: Quản lý thiết bị, người dùng, hệ thống.
  - ◆ *User*: Chỉ điều khiển thiết bị và xem dữ liệu của mình.
- **Bảo mật ứng dụng :**
  - Token đăng nhập (JWT) lưu tạm, tự hết hạn sau 24h.
  - Hạn chế tấn công brute-force hoặc truy cập trái phép

### 3. Tính ổn định và tin cậy (Reliability & Availability)

- **Hoạt động liên tục 24/7:**
  - Tự động khởi động lại khi mất điện hoặc reset.
- **Cơ chế tự phục hồi:**
  - Khi mất WiFi, ESP32 tự động reconnect sau 10 giây.
  - Server có cơ chế hàng đợi tin nhắn (MQTT retained message).
- **Khả năng xử lý lỗi:**
  - Nếu relay hoặc cảm biến không phản hồi → hệ thống gửi cảnh báo lỗi “Device Error” đến app.
- **Độ tin cậy dữ liệu**
  - Dữ liệu cảm biến được xác thực checksum trước khi lưu vào database.

### 4. Khả năng mở rộng (Scalability & Extensibility)

- Có thể thêm cảm biến hoặc thiết bị mới (đèn, rèm, camera, cảm biến mới) mà không cần thay đổi cấu trúc hệ thống.
- MQTT broker hỗ trợ nhiều node ESP32 cùng kết nối.
- Server thiết kế dạng microservice hoặc RESTful API, dễ mở rộng khi có nhiều người dùng.
- Có thể tích hợp AI hoặc học máy trong tương lai (ví dụ: dự đoán thói quen bật đèn).

### 5. Tính thân thiện và dễ sử dụng (Usability)

- **Giao diện dashboard/app:**
  - Thiết kế trực quan, biểu đồ rõ ràng (line chart, gauge).
  - Có màu sắc trực quan cho cảnh báo (đỏ: nguy hiểm, vàng: cảnh báo, xanh: bình thường).
- **Hỗ trợ điều khiển giọng nói:**

- Tích hợp API Google Assistant hoặc lệnh tiếng Việt (ví dụ: “Bật đèn phòng khách”).
- **Ngôn ngữ hiển thị:**
  - Giao diện hỗ trợ tiếng Việt cho người dùng phổ thông.

## V. Phân tích ràng buộc

- Phần cứng : Số chân GPIO giới hạn trên ESP32
- Kết nối mạng : Hệ thống phụ thuộc vào WiFi/Internet.
- Nguồn điện : Cần nguồn ổn định cho cảm biến và relay
- Chí phí : Giới hạn kinh phí trong phạm vi
- Thời gian : Thực hiện trong 4–8 tuần theo kế hoạch môn học

## VI. Mô hình hóa yêu cầu

### 1. Mô hình tổng thể hệ thống

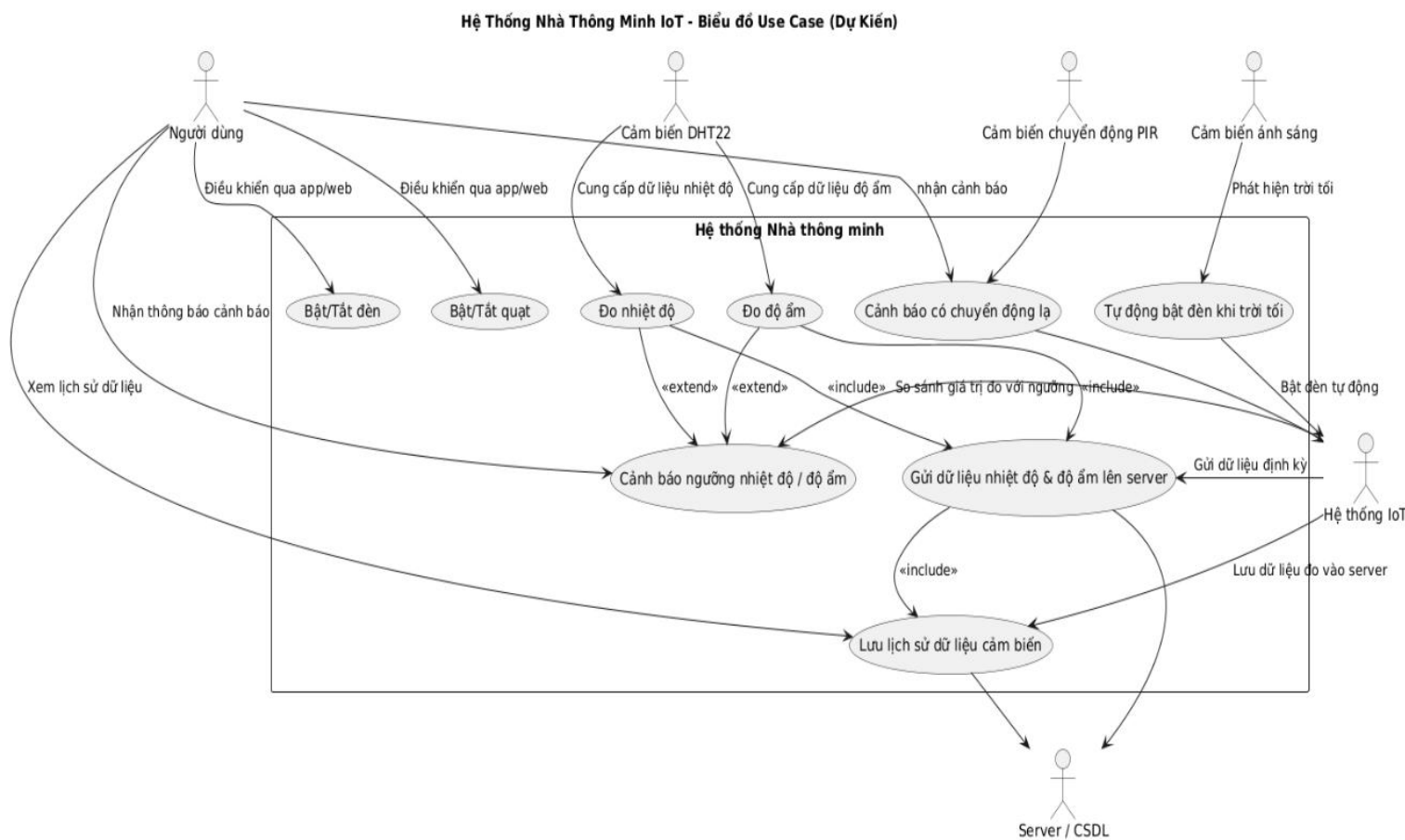
Hệ thống Nhà thông minh được thiết kế theo kiến trúc ba tầng:

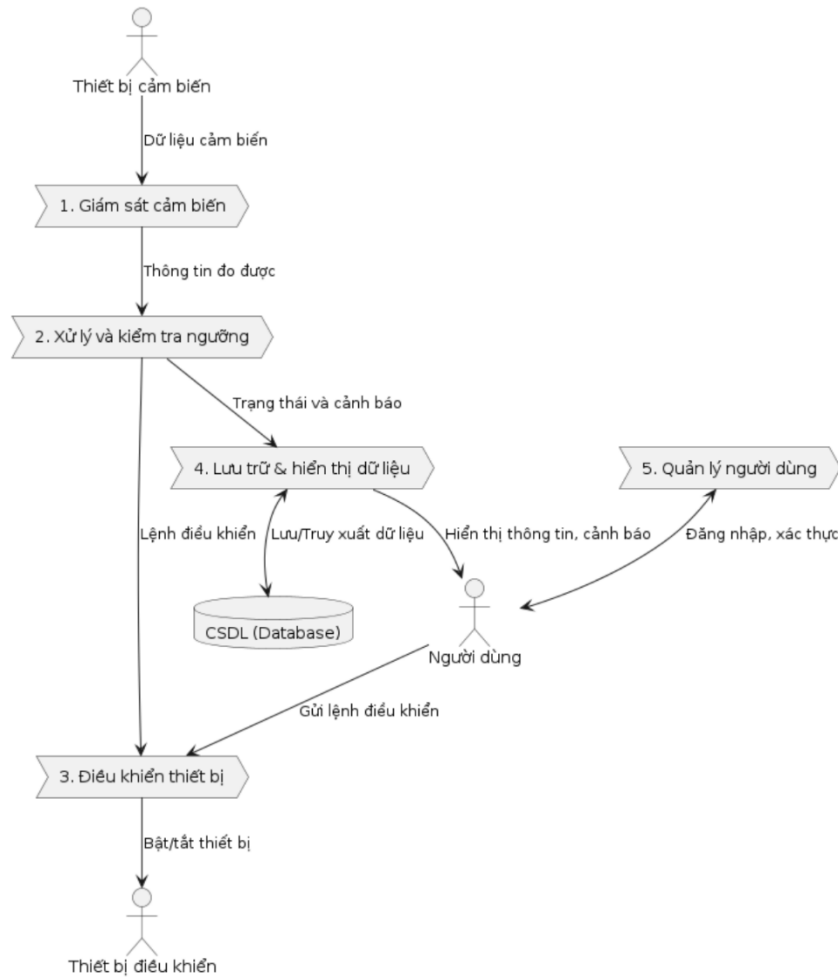
- **Tầng cảm biến (Sensor Layer):** Bao gồm các cảm biến DHT22 (nhiệt độ, độ ẩm), LDR (ánh sáng) và các relay điều khiển thiết bị.
- **Tầng điều khiển (Controller Layer):** Vi điều khiển ESP32 thu thập dữ liệu, xử lý logic cơ bản và truyền thông với máy chủ qua giao thức MQTT hoặc HTTP.
- **Tầng ứng dụng (Application Layer):** Gồm server backend, cơ sở dữ liệu và giao diện web/app giúp người dùng theo dõi, điều khiển, nhận thông báo.

### 2. Biểu đồ Use Case tổng quát

**Tác nhân chính:**

- **Chủ hộ:** theo dõi, điều khiển thiết bị, nhận cảnh báo.
- **Quản trị viên:** cấu hình, giám sát và bảo trì hệ thống.





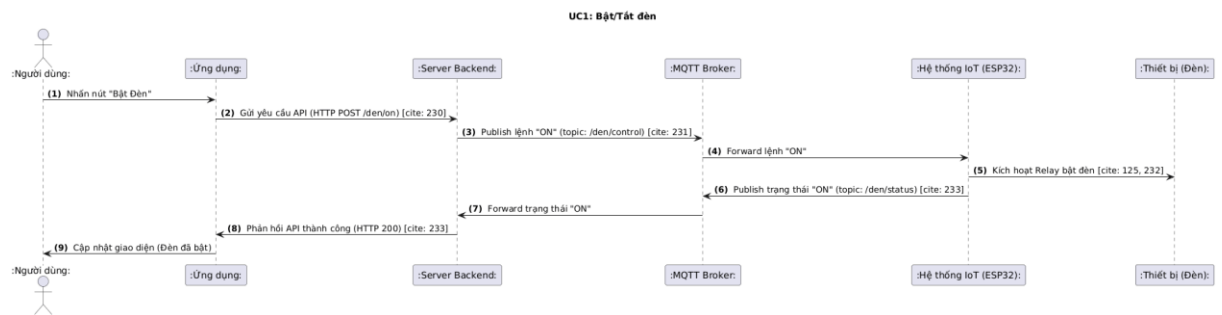
#### 4. Mô hình kiến trúc triển khai

- **Thiết bị IoT:** ESP32 với các cảm biến/relay.
- **Server Backend:** PHP xử lý API và MQTT message.
- **Cơ sở dữ liệu:** MySQL lưu trữ dữ liệu cảm biến & người dùng.
- **Giao diện người dùng:** Web giao tiếp qua REST API

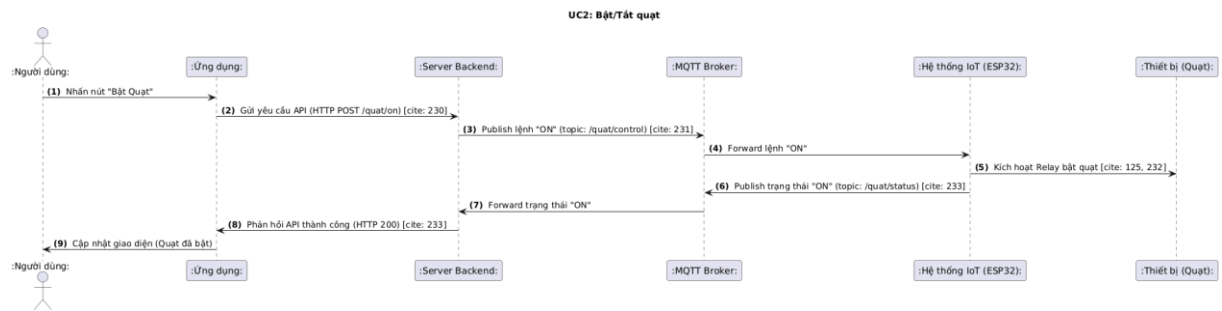
## D. Phân tích thiết kế

### I. Biểu đồ tuần tự (Sequence Diagram)

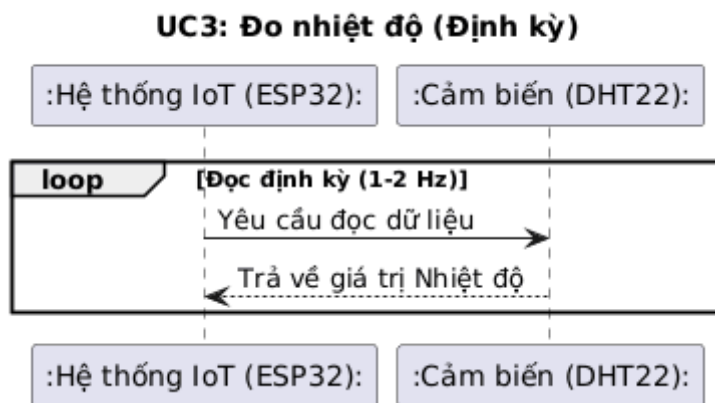
- ❖ UC1(Bật/Tắt đèn):



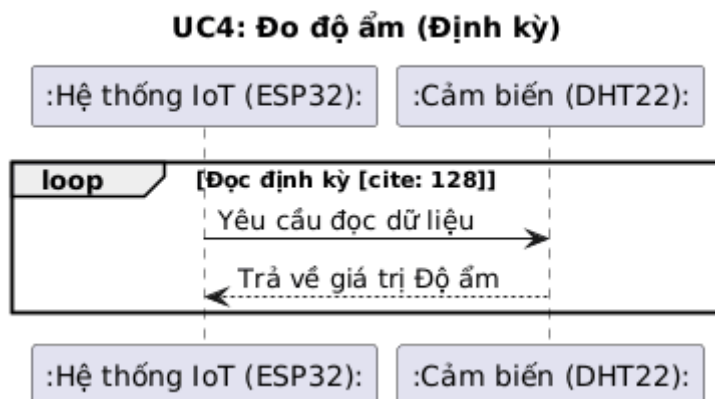
❖ UC2(Bật/Tắt quạt):



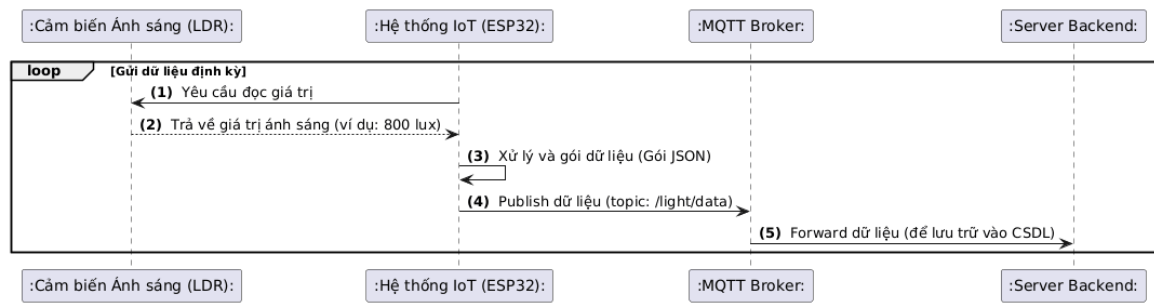
❖ UC3(Đo nhiệt độ):



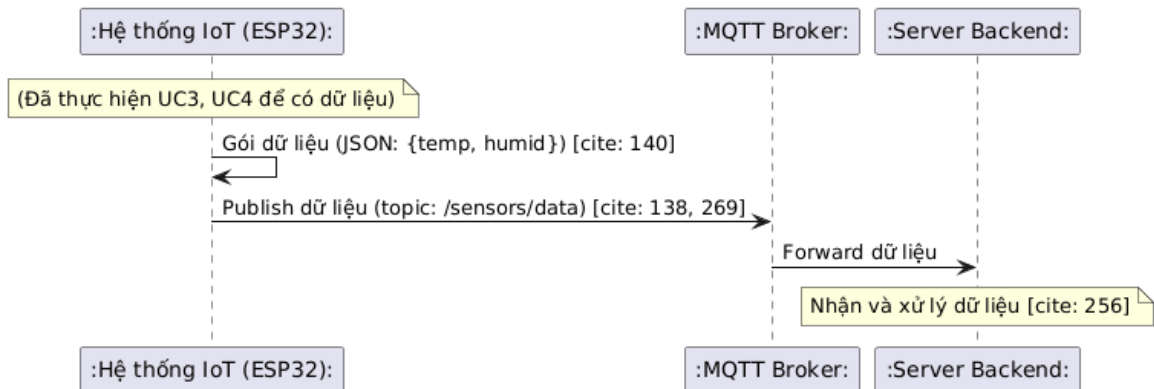
❖ UC4(Đo độ ẩm):



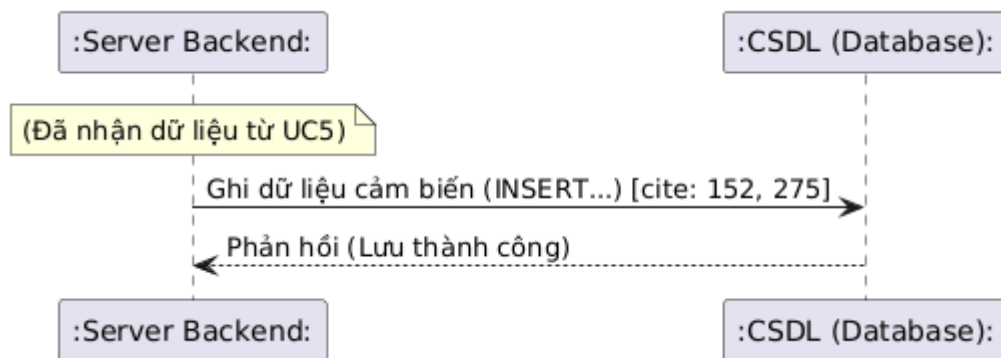
❖ UC5(Đo ánh sáng):



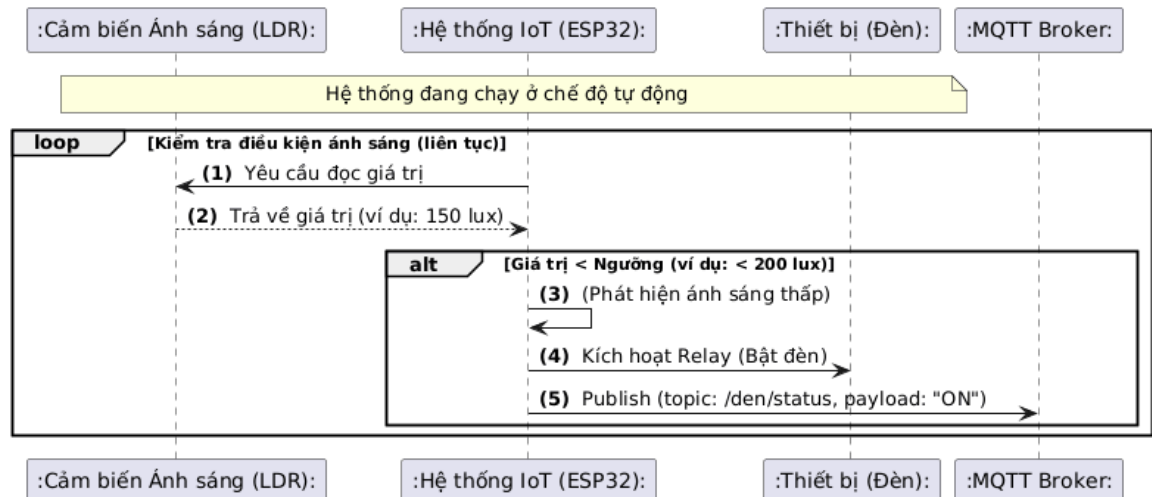
❖ UC6(Gửi dữ liệu nhiệt độ, độ ẩm và ánh sáng lên server):



❖ UC7(Lưu lịch sử dữ liệu cảm biến):

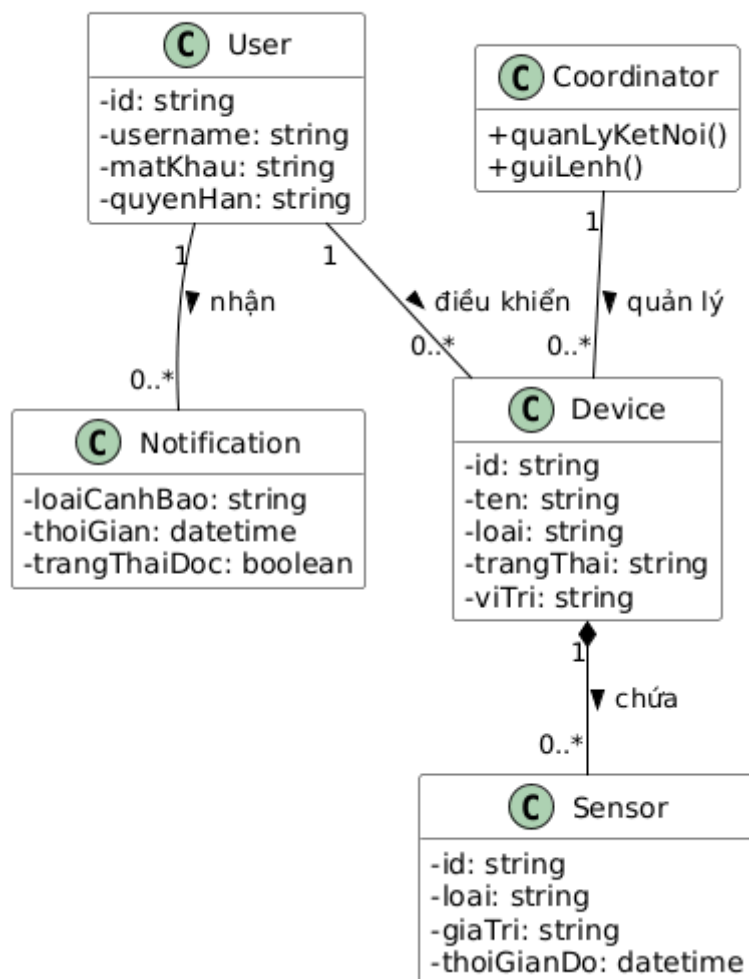


❖ UC8(Tự động bật đèn khi trời tối):



## II. Biểu đồ lớp (Class Diagram)

**Biểu đồ Lớp Hệ Thống Nhà Thông Minh**



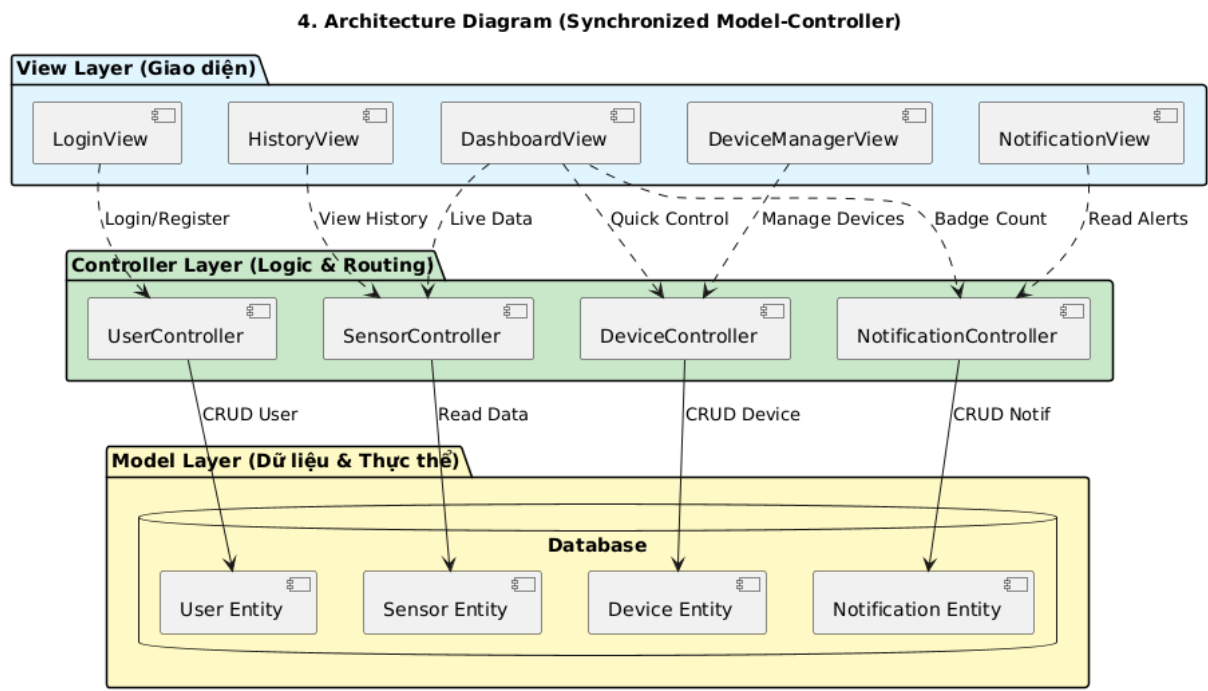
Các lớp chính trong hệ thống:

- **Device**: id, tên, loại, trạng thái, vị trí.
- **Sensor**: id, loại, giá trị, thời gian đo.
- **User**: id, username, mật khẩu, quyền hạn.
- **Coordinator**: quản lý kết nối và gửi lệnh.
- **Notification**: loại cảnh báo, thời gian, trạng thái đọc.

Mối quan hệ:

- User → điều khiển nhiều Device.
- Device → có thể chứa nhiều Sensor.
- Coordinator → quản lý Device và giao tiếp với Server.

III. Thiết kế MVC



❖ Tầng Model:

| Tên Lớp (Entity) | Mô tả Chức năng                                                                                    | Thuộc tính chính                                                   |
|------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
| User             | Đại diện cho người dùng hệ thống. Lưu trữ thông tin định danh và xác thực.                         | user_id, username, password_hash, email, role.                     |
| Device           | Đại diện cho một thiết bị IoT vật lý (đèn, quạt, công tắc). Chứa trạng thái hiện tại của thiết bị. | device_id, name, type, status (ON/OFF), user_id (chủ sở hữu).      |
| Sensor           | Đại diện cho cảm biến gắn trên thiết bị hoặc độc lập. Lưu trữ giá trị môi trường đo đạc được.      | sensor_id, type (nhiệt độ, độ ẩm...), current_value, last_updated. |
| Notification     | Đại diện cho các thông báo/cảnh báo gửi đến người dùng (ví dụ: nhiệt độ cao).                      | notification_id, message, is_read (đã đọc chưa), timestamp.        |

❖ Tầng Controller:

- **UserController:**

- **Vai trò:** Quản lý toàn bộ các tác vụ liên quan đến tài khoản người dùng.
- **Chức năng chính:**
  - Xử lý đăng nhập (login), đăng ký (register).
  - Xác thực phiên làm việc (Session/Token).
  - Cập nhật thông tin hồ sơ (updateProfile).
- **Tương tác:** Nhận yêu cầu từ LoginView, UserProfileView -> Thao tác với Model User.

- **DeviceController**

- **Vai trò:** Quản lý vòng đời và điều khiển thiết bị.
- **Chức năng chính:**
  - Lấy danh sách thiết bị của một user (getAllDevices).
  - Gửi lệnh điều khiển xuống phần cứng (ví dụ: Bật đèn, Tắt quạt - sendCommand).
  - Thêm mới hoặc xóa thiết bị (addDevice, deleteDevice).
- **Tương tác:** Nhận yêu cầu từ DeviceManagerView và DashboardView -> Thao tác với Model Device (và gửi MQTT).

- **SensorController**

- **Vai trò:** Quản lý dữ liệu cảm biến và lịch sử đo đạc.
- **Chức năng chính:**
  - Lấy thông số cảm biến mới nhất để hiển thị realtime (getSensorData).
  - Truy xuất dữ liệu lịch sử theo khoảng thời gian để vẽ biểu đồ (getHistory).
- **Tương tác:** Nhận yêu cầu từ DashboardView (xem nhanh) và HistoryView (xem chi tiết) -> Thao tác với Model Sensor.

- **NotificationController**

- **Vai trò:** Quản lý luồng thông báo hệ thống.
- **Chức năng chính:**
  - Lấy danh sách thông báo chưa đọc (getUnread).
  - Đánh dấu đã đọc (markAsRead) hoặc xóa thông báo.
- **Tương tác:** Nhận yêu cầu từ NotificationView -> Thao tác với Model Notification.

❖ Tầng View:

- **DashboardView (Trang chủ)**

- **Mô tả:** Màn hình tổng quan, là "trung tâm chỉ huy" của người dùng.
- **Đặc điểm xử lý:** Vì đây là màn hình tổng hợp, nó sẽ **gọi đến nhiều Controller khác nhau**:
  - Gọi DeviceController.getAllDevices() để hiển thị các nút bật/tắt nhanh.
  - Gọi SensorController.getSensorData() để hiển thị thể nhiệt độ/độ ẩm hiện tại.
  - Gọi NotificationController để hiển thị số lượng tin báo mới (badge).

- **LoginView (Đăng nhập)**

- **Mô tả:** Giao diện để người dùng nhập Username/Password.
- **Đặc điểm xử lý:** Gọi duy nhất UserController.login() để xác thực.

- **DeviceManagerView (Quản lý thiết bị)**

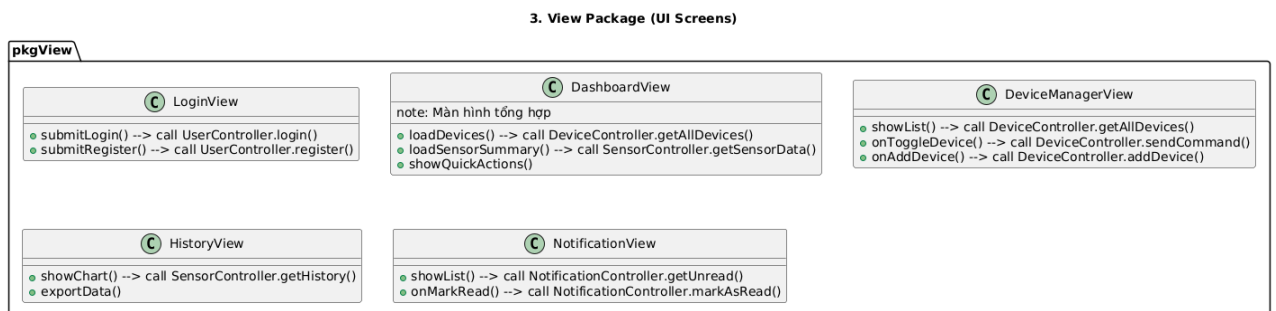
- **Mô tả:** Màn hình danh sách chi tiết, cho phép thêm/sửa/xóa thiết bị.
- **Đặc điểm xử lý:** Gọi DeviceController để thực hiện các lệnh CRUD (Create, Read, Update, Delete) thiết bị.

- **HistoryView (Lịch sử cảm biến)**

- **Mô tả:** Giao diện hiển thị biểu đồ đường (Line Chart) hoặc bảng số liệu lịch sử.

- **Đặc điểm xử lý:** Cho phép user chọn ngày tháng, sau đó gọi `SensorController.getHistory()` để lấy mảng dữ liệu và vẽ lên biểu đồ.
- **NotificationView (Thông báo)**
  - **Mô tả:** Danh sách các cảnh báo an ninh hoặc thông tin hệ thống.
  - **Đặc điểm xử lý:** Gọi `NotificationController` để tải danh sách và cập nhật trạng thái đã đọc.

## 1. Thiết kế chi tiết từng View



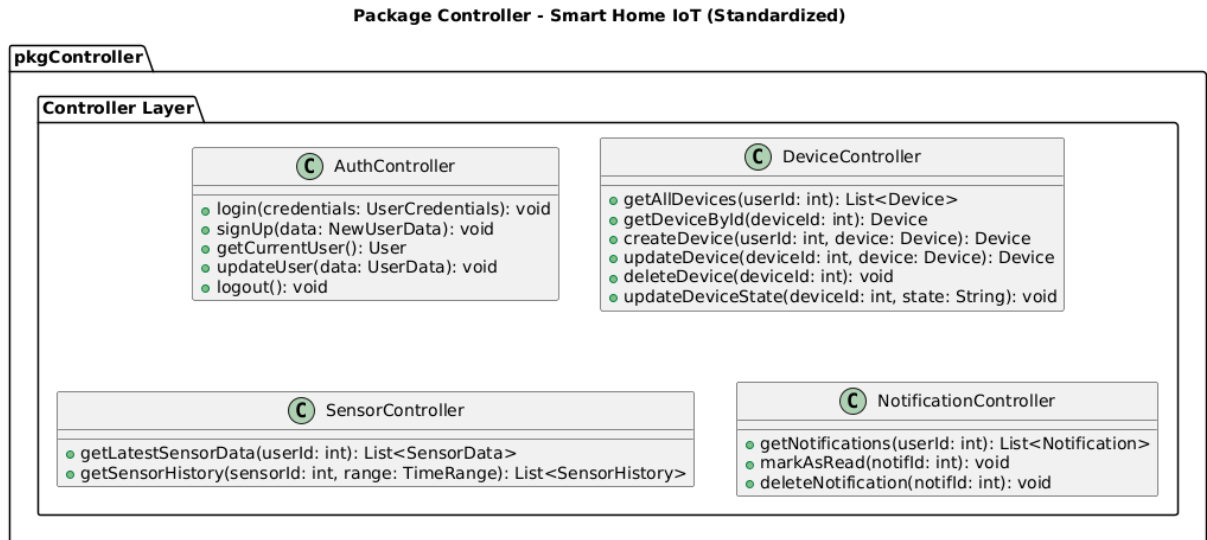
- **LoginView (Màn hình Đăng nhập):** Đây là cổng vào của ứng dụng, đảm bảo tính bảo mật và xác thực người dùng.
  - **Vai trò:** Giao diện xác thực danh tính.
  - **Thành phần giao diện (UI Components):**
    - Ô nhập liệu (Input field): Tên đăng nhập (username), Mật khẩu (password).
    - Nút bấm (Button): "Đăng nhập", "Đăng ký", "Quên mật khẩu".
  - **Xử lý logic giao diện:**
    - Kiểm tra tính hợp lệ cơ bản của dữ liệu nhập (ví dụ: không để trống, đúng định dạng email).
    - Gọi `UserController.login()` khi người dùng nhấn nút Đăng nhập.
    - Hiện thị thông báo lỗi nếu đăng nhập thất bại hoặc chuyển hướng sang `DashboardView` nếu thành công.
- **DashboardView (Màn hình Trang chủ):** Đây là màn hình quan trọng nhất, đóng vai trò là "Trung tâm điều khiển". Nó tổng hợp thông tin từ nhiều nguồn khác nhau.

- **Vai trò:** Hiển thị cái nhìn tổng quan về trạng thái ngôi nhà và cung cấp các thao tác nhanh.
- **Thành phần giao diện:**
  - **Widget cảm biến:** Hiển thị nhiệt độ, độ ẩm hiện tại (Lấy dữ liệu từ SensorController).
  - **Danh sách thiết bị yêu thích:** Các nút bật/tắt nhanh cho đèn phòng khách, quạt (Gọi DeviceController).
  - **Biểu tượng thông báo:** Hiển thị số lượng cảnh báo chưa đọc (Gọi NotificationController).
- **Xử lý logic giao diện:**
  - Tự động làm mới dữ liệu (Auto-refresh) sau mỗi khoảng thời gian (ví dụ: 5 giây) để cập nhật trạng thái thời gian thực.
  - Gửi lệnh bật/tắt ngay lập tức mà không cần truy cập vào trang chi tiết.
- **DeviceManagerView (Màn hình Quản lý Thiết bị):** Màn hình này dùng để quản lý sâu hơn về danh sách và cấu hình các thiết bị.
- **Vai trò:** Cung cấp đầy đủ các chức năng CRUD (Thêm, Xem, Sửa, Xóa) cho thiết bị.
- **Thành phần giao diện:**
  - Danh sách cuộn (List/Grid view) hiển thị toàn bộ thiết bị, phân loại theo phòng (Phòng ngủ, Phòng khách...).
  - Nút "Thêm thiết bị mới" (Floating Action Button).
  - Modal/Popup để nhập thông tin thiết bị mới.
- **Xử lý logic giao diện:**
  - Gọi DeviceController.getAllDevices() khi tải trang.
  - Khi người dùng thêm thiết bị: Thu thập dữ liệu từ form -> Gọi DeviceController.addDevice().
  - Khi người dùng xóa thiết bị: Hiển thị hộp thoại xác nhận -> Gọi DeviceController.deleteDevice().

- **HistoryView (Màn hình Lịch sử & Báo cáo):** Dành cho việc theo dõi, phân tích dữ liệu môi trường theo thời gian.
  - **Vai trò:** Trực quan hóa dữ liệu cảm biến dưới dạng biểu đồ hoặc bảng số liệu.
  - **Thành phần giao diện:**
    - Bộ lọc thời gian (Date Picker): Hôm nay, Tuần này, Tháng này.
    - Dropdown menu: Chọn loại cảm biến cần xem (Nhiệt độ, Độ ẩm, CO2...).
    - Biểu đồ (Chart): Biểu đồ đường (Line chart) để thể hiện xu hướng tăng giảm.
  - **Xử lý logic giao diện:**
    - Lắng nghe sự kiện thay đổi bộ lọc thời gian.
    - Gọi `SensorController.getHistory(sensorId, fromDate, toDate)` để lấy mảng dữ liệu.
    - Render (vẽ) dữ liệu trả về lên biểu đồ.
- **NotificationView (Màn hình Thông báo):** Nơi lưu trữ các cảnh báo an ninh và nhật ký hệ thống.
  - **Vai trò:** Giúp người dùng xem lại các sự kiện đã xảy ra (phát hiện chuyển động, nhiệt độ quá cao...).
  - **Thành phần giao diện:**
    - Danh sách các thẻ thông báo, được sắp xếp theo thời gian (mới nhất ở trên).
    - Đánh dấu màu sắc khác nhau cho thông báo "Chưa đọc" và "Đã đọc".
    - Nút "Đánh dấu tất cả là đã đọc".
  - **Xử lý logic giao diện:**
    - Gọi `NotificationController.getUnread()` để tải danh sách.

- Khi người dùng bấm vào một thông báo: Chuyển trạng thái hiển thị sang "Đã đọc" và gọi `NotificationController.markAsRead()` để cập nhật xuống Database.

## 2. Thiết kế chi tiết tầng Controller



❖ **Class AuthController (Bộ điều khiển Xác thực & Người dùng):** Lớp này chịu trách nhiệm quản lý định danh, bảo mật và thông tin cá nhân của người dùng. Nó gộp cả chức năng xác thực (Authentication) và quản lý người dùng (User Management).

- **login(credentials: UserCredentials): void**
  - **Chức năng:** Tiếp nhận tên đăng nhập và mật khẩu. Kiểm tra tính hợp lệ, nếu đúng sẽ cấp phát phiên làm việc (Session) hoặc Token (JWT).
- **signUp(data: NewUserData): void**
  - **Chức năng:** Đăng ký tài khoản mới. Kiểm tra xem username/email đã tồn tại chưa trước khi tạo user mới trong DB.
- **getCurrentUser(): User**
  - **Chức năng:** Lấy thông tin của người dùng đang đăng nhập hiện tại (dựa trên Token/Session) để hiển thị lên Profile hoặc Header.
- **updateUser(data: UserData): void**
  - **Chức năng:** Cập nhật thông tin cá nhân như đổi tên hiển thị, đổi mật khẩu, cập nhật email.

- **logout(): void**
  - **Chức năng:** Hủy phiên làm việc hiện tại, xóa Token khỏi bộ nhớ tạm để đăng xuất an toàn.
  
- ❖ **Class DeviceController (Bộ điều khiển Thiết bị):** Đây là lớp quan trọng nhất trong hệ thống IoT, chịu trách nhiệm quản lý vòng đời của thiết bị và điều khiển trạng thái hoạt động của chúng.
  - **getAllDevices(userId: int): List<Device>**
    - **Chức năng:** Lấy danh sách toàn bộ thiết bị mà người dùng (userId) sở hữu. Thường dùng để hiển thị lên Dashboard hoặc danh sách quản lý.
  - **getDeviceById(deviceId: int): Device**
    - **Chức năng:** Xem chi tiết thông tin của một thiết bị cụ thể.
  - **createDevice(userId: int, device: Device): Device**
    - **Chức năng:** Thêm một thiết bị mới vào hệ thống và gán nó cho người dùng.
  - **updateDevice(deviceId: int, device: Device): Device**
    - **Chức năng:** Cập nhật **thông tin mô tả** của thiết bị (Ví dụ: Đổi tên thiết bị từ "Đèn 1" thành "Đèn Phòng Ngủ", đổi vị trí lắp đặt).
    - *Lưu ý:* Hàm này khác với hàm điều khiển trạng thái bên dưới.
  - **deleteDevice(deviceId: int): void**
    - **Chức năng:** Xóa thiết bị khỏi hệ thống (Unpair).
  - **updateDeviceState(deviceId: int, state: String): void**
    - **Chức năng:** Gửi lệnh điều khiển **vật lý** xuống thiết bị (Ví dụ: Gửi lệnh "ON", "OFF", hoặc set nhiệt độ "25"). Đây là hàm kích hoạt hành động thực tế của thiết bị IoT.
  
- ❖ **Class SensorController (Bộ điều khiển Cảm biến):** Lớp này chuyên trách việc thu thập và phân phối dữ liệu môi trường. Lưu ý rằng Controller này thường chỉ

có chức năng **Đọc (Read)** vì dữ liệu cảm biến là do thiết bị gửi lên, người dùng ít khi can thiệp sửa đổi.

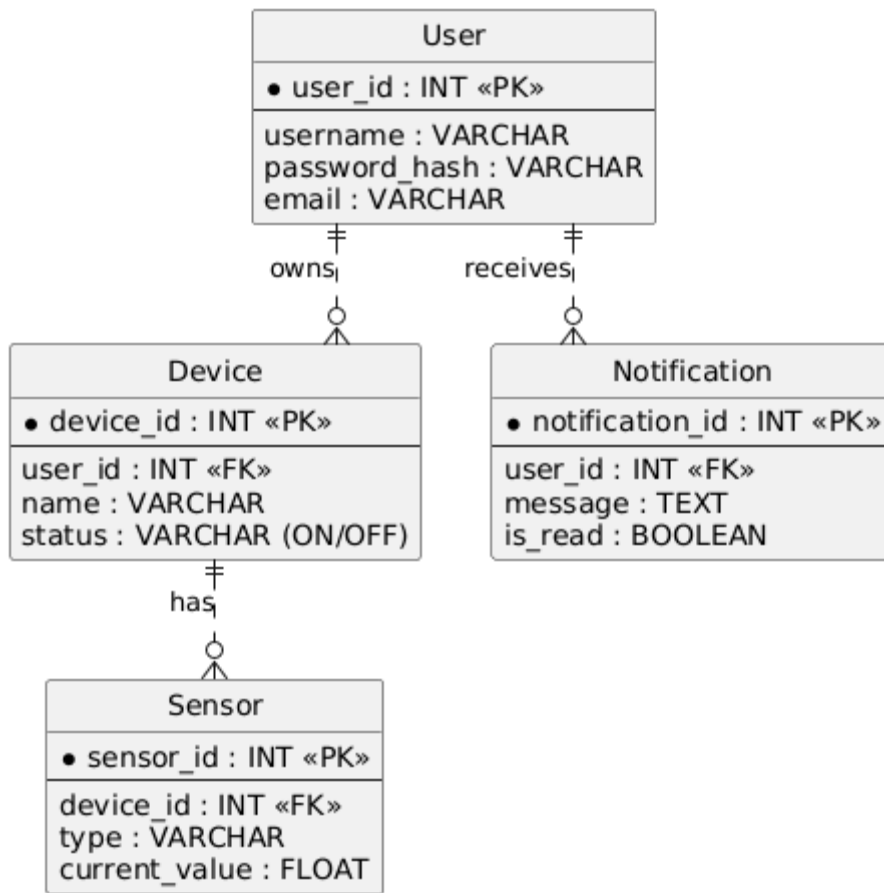
- **getLatestSensorData(userId: int): List<SensorData>**
  - **Chức năng:** Lấy giá trị mới nhất của tất cả cảm biến thuộc về user đó. Dùng để hiển thị trạng thái thời gian thực (Real-time) trên Dashboard (VD: Nhiệt độ hiện tại là 30°C).
- **getSensorHistory(sensorId: int, range: TimeRange): List<SensorHistory>**
  - **Chức năng:** Truy xuất lịch sử dữ liệu của một cảm biến cụ thể trong khoảng thời gian (range). Dùng để vẽ biểu đồ thống kê hoặc phân tích xu hướng.

❖ **Class NotificationController (Bộ điều khiển Thông báo):** Quản lý luồng thông tin cảnh báo từ hệ thống đến người dùng.

- **getNotifications(userId: int): List<Notification>**
  - **Chức năng:** Lấy danh sách các thông báo (cảnh báo cháy, chuyển động, thiết bị mất kết nối...) dành cho user. Thường sẽ ưu tiên lấy thông báo chưa đọc hoặc mới nhất.
- **markAsRead(notifId: int): void**
  - **Chức năng:** Đánh dấu một thông báo cụ thể là "Đã xem" để tắt đèn báo hiệu hoặc ẩn khỏi danh sách chưa đọc.
- **deleteNotification(notifId: int): void**
  - **Chức năng:** Xóa vĩnh viễn một thông báo khỏi danh sách lưu trữ.

#### IV. Thiết kế cơ sở dữ liệu

## 1. Database ERD (Models)



❖ **Entity User (Người dùng):** Đây là bảng trung tâm lưu trữ thông tin tài khoản của người sử dụng hệ thống.

- **Vai trò:** Quản lý danh tính và quyền truy cập. Mọi dữ liệu khác (thiết bị, thông báo) đều xoay quanh người dùng này.
- **Các thuộc tính (Attributes):**
  - **user\_id (INT, PK):** Khóa chính, mã định danh duy nhất cho mỗi người dùng.
  - **username (VARCHAR):** Tên đăng nhập vào hệ thống.
  - **password\_hash (VARCHAR):** Mật khẩu đã được mã hóa (Lưu ý: Không lưu mật khẩu thô plain-text để đảm bảo bảo mật).
  - **email (VARCHAR):** Địa chỉ email dùng để liên lạc hoặc khôi phục mật khẩu.
- **Mối quan hệ:**

- **1-n với Device:** Một User có thể sở hữu nhiều Device.
- **1-n với Notification:** Một User có thể nhận nhiều Notification.
- ❖ **Entity Device (Thiết bị):** Đại diện cho các thiết bị phần cứng IoT vật lý được kết nối vào hệ thống (ví dụ: Công tắc thông minh, Quạt, Đèn, Bộ điều khiển trung tâm).
  - **Vai trò:** Quản lý thông tin và trạng thái hoạt động của thiết bị.
  - **Các thuộc tính (Attributes):**
    - device\_id (INT, **PK**): Khóa chính, mã định danh duy nhất của thiết bị.
    - user\_id (INT, **FK**): Khóa ngoại liên kết với bảng User. Xác định thiết bị này thuộc về ai.
    - name (VARCHAR): Tên gọi nhớ do người dùng đặt (VD: "Đèn phòng ngủ", "Quạt trần").
    - status (VARCHAR): Trạng thái hiện tại của thiết bị (VD: "ON", "OFF", "OFFLINE").
  - **Mối quan hệ:**
    - **n-1 với User:** Thuộc về một User cụ thể.
    - **1-n với Sensor:** Một Device (ví dụ: bo mạch ESP32) có thể gắn kèm nhiều Sensor (cảm biến nhiệt, cảm biến ẩm...).
  - **Entity Sensor (Cảm biến)**

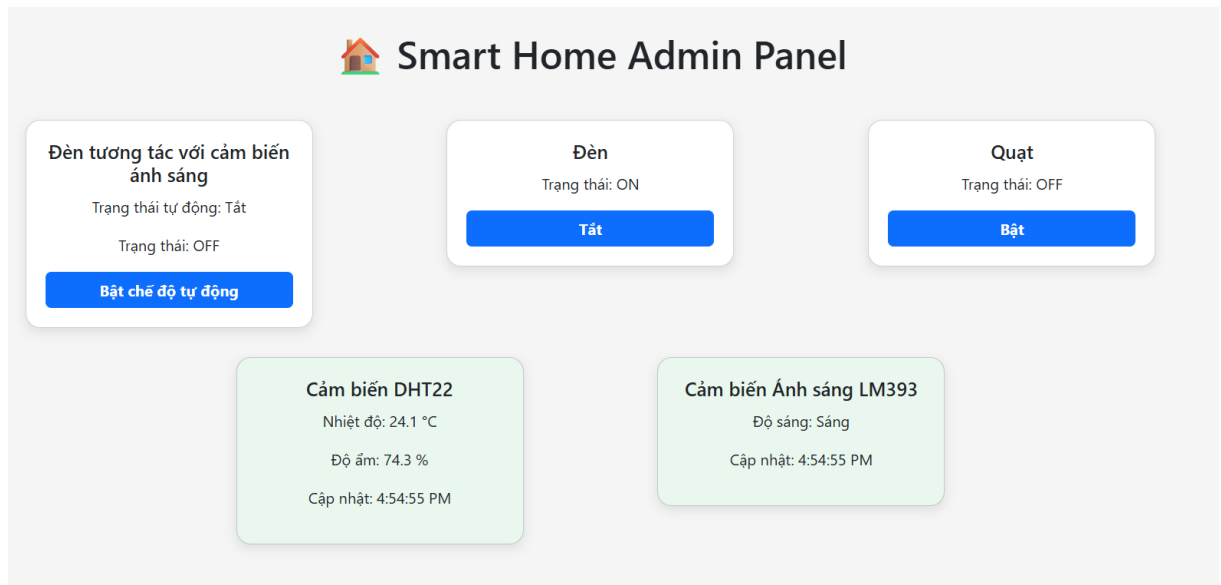
Đại diện cho các đầu đo đặc môi trường hoặc các thông số kỹ thuật.

- **Vai trò:** Lưu trữ giá trị thời gian thực mà thiết bị đo đạc được.
- **Các thuộc tính (Attributes):**
  - sensor\_id (INT, **PK**): Khóa chính của cảm biến.
  - device\_id (INT, **FK**): Khóa ngoại liên kết với bảng Device. Cho biết cảm biến này được gắn trên thiết bị phần cứng nào.
  - type (VARCHAR): Loại dữ liệu đo (VD: "Temperature", "Humidity", "CO2").

- `current_value` (FLOAT): Giá trị đo đặc hiện tại (VD: 25.5, 60.0). Dữ liệu này thường xuyên thay đổi.
- **Mối quan hệ:**
  - **n-1 với Device:** Thuộc về một Device cụ thể.
- ❖ **Entity Notification (Thông báo):** Lưu trữ các tin nhắn cảnh báo hoặc thông tin hệ thống gửi đến người dùng.
- **Vai trò:** Kênh giao tiếp một chiều từ hệ thống tới người dùng.
- **Các thuộc tính (Attributes):**
  - `notification_id` (INT, **PK**): Khóa chính của thông báo.
  - `user_id` (INT, **FK**): Khóa ngoại liên kết với bảng User. Xác định thông báo này gửi cho ai.
  - `message` (TEXT): Nội dung chi tiết của thông báo (VD: "Cảnh báo: Nhiệt độ phòng khách quá cao!").
  - `is_read` (BOOLEAN): Cờ trạng thái, false nghĩa là chưa đọc, true là đã đọc. Dùng để hiển thị chấm đỏ (badge) trên giao diện.
- **Mối quan hệ:**
  - **n-1 với User:** Được gửi tới một User cụ thể.

## E. KẾT QUẢ ĐẠT ĐƯỢC

### 1. Giao diện web điều khiển

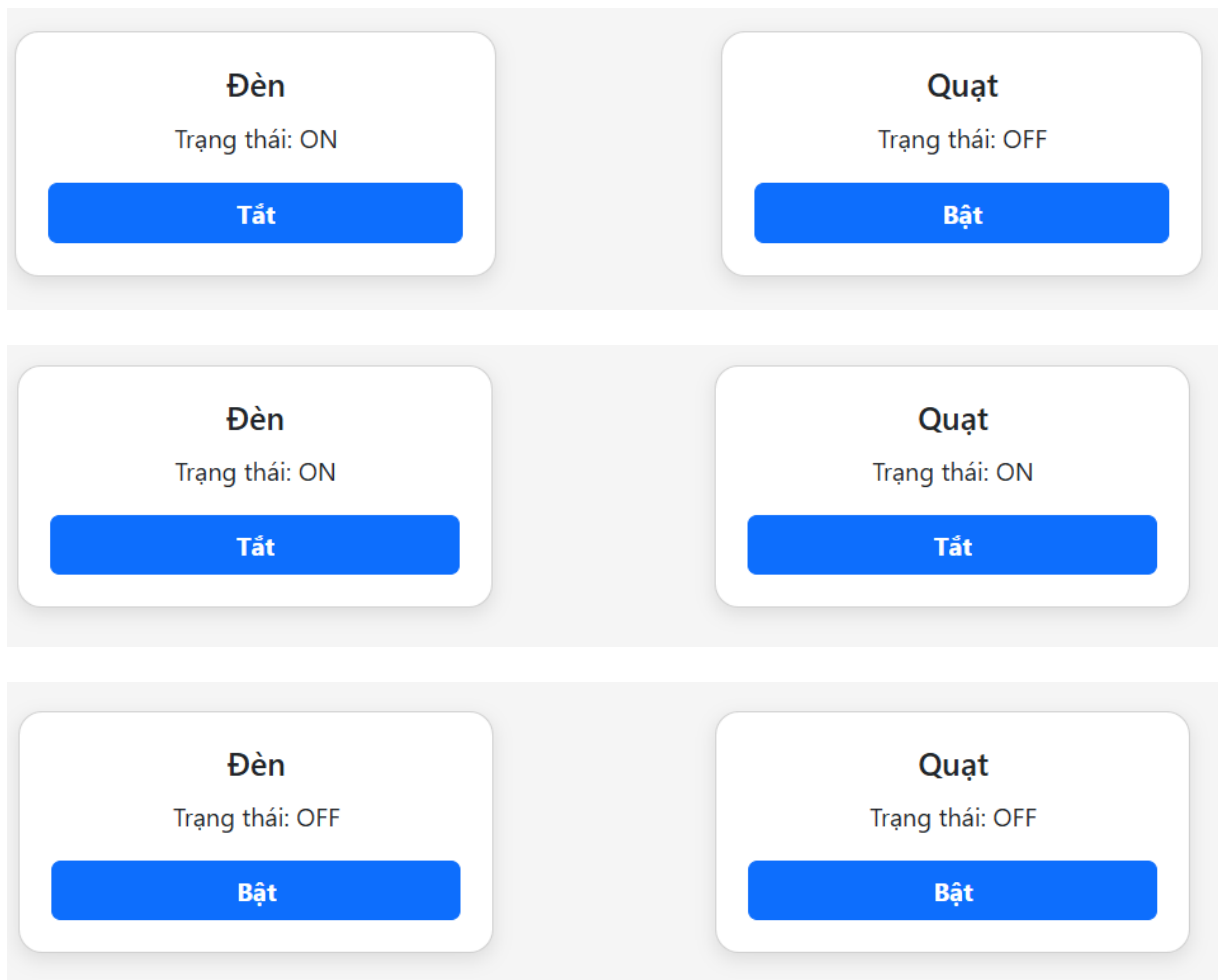


Kết quả giao diện Web điều khiển (Dashboard)

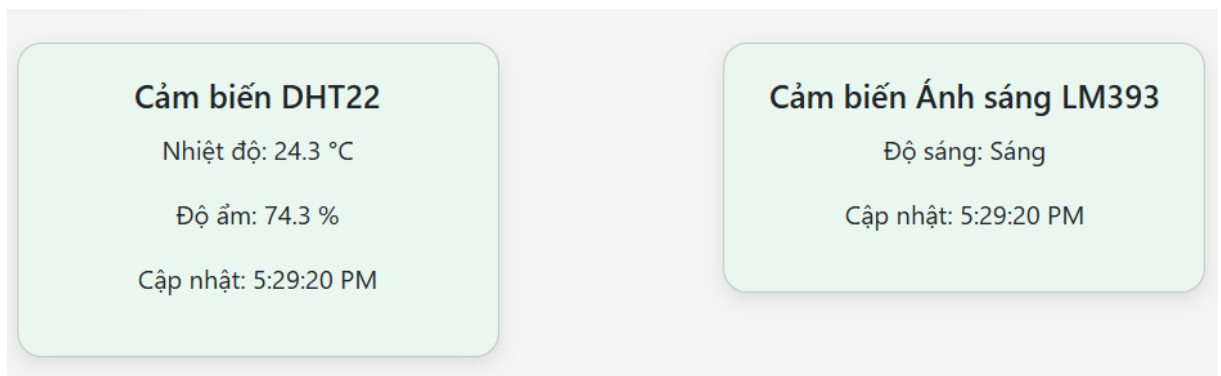
- Hiện thị realtime giá trị: Nhiệt độ – độ ẩm, ánh sáng.
- Cho phép bật tắt đèn/quạt trực tiếp trên giao diện.
- Cho phép bật tắt chế độ tự động của đèn với cảm biến ánh sáng

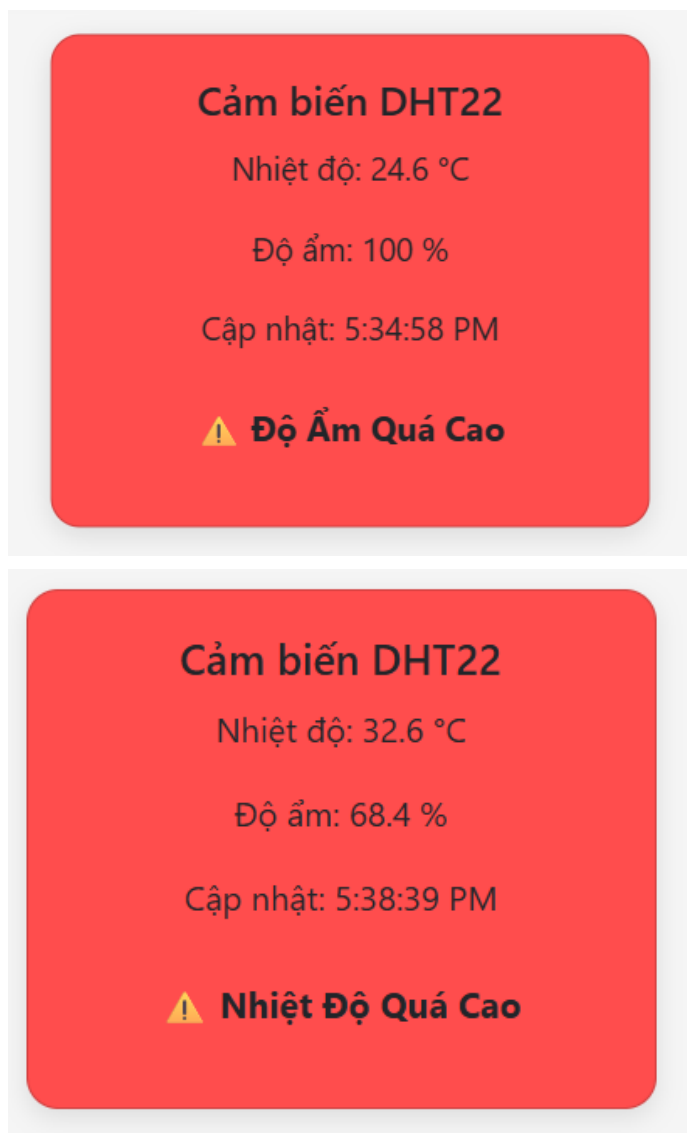
### 2. Điều khiển thiết bị từ xa

Cho phép bật tắt quạt, đèn trực tiếp trên web



### 3. Cảm biến gửi dữ liệu lên web liên tục và gửi cảnh báo khi vượt ngưỡng





#### 4. Hiện thị lịch sử người dùng

| Lịch sử người dùng  |                 |                      |           |            |
|---------------------|-----------------|----------------------|-----------|------------|
| Thời gian           | Người dùng      | Thiết bị             | Hành động | Trạng thái |
| 20:31:08 26/11/2025 | Nguyễn Duy Hiếu | Đèn cảm ứng ánh sáng | Tắt       | OFF        |
| 20:24:08 26/11/2025 | Nguyễn Duy Hiếu | Đèn                  | Bật       | ON         |
| 20:23:08 26/11/2025 | Nguyễn Duy Hiếu | Quạt                 | Bật       | ON         |
| 20:20:08 26/11/2025 | Nguyễn Duy Hiếu | Quạt                 | Tắt       | OFF        |
| 19:44:08 26/11/2025 | Nguyễn Duy Hiếu | Quạt                 | Bật       | ON         |
| 19:26:08 26/11/2025 | Nguyễn Duy Hiếu | Đèn                  | Bật       | OFF        |
| 19:10:08 26/11/2025 | Nguyễn Duy Hiếu | Đèn cảm ứng ánh sáng | Tắt       | ON         |
| 19:09:08 26/11/2025 | Nguyễn Duy Hiếu | Đèn                  | Bật       | ON         |
| 18:59:08 26/11/2025 | Nguyễn Duy Hiếu | Đèn                  | Tắt       | OFF        |

#### 5. Nhật ký nhiệt độ, độ ẩm và ánh sáng

| Lịch Sử Cảm Biến    |               |           |          |
|---------------------|---------------|-----------|----------|
| Thời gian           | Nhiệt độ (°C) | Độ ẩm (%) | Ánh sáng |
| 21:46:47 26/11/2025 | 23.7          | 72.4      | Tối      |
| 21:46:37 26/11/2025 | 24.1          | 71.4      | Tối      |
| 21:46:27 26/11/2025 | 24.3          | 72.1      | Tối      |
| 21:46:17 26/11/2025 | 24.6          | 72.1      | Tối      |
| 21:46:07 26/11/2025 | 24.6          | 72.2      | Sáng     |
| 21:45:57 26/11/2025 | 24.3          | 72.6      | Sáng     |
| 21:45:47 26/11/2025 | 24.6          | 72.0      | Sáng     |
| 21:45:37 26/11/2025 | 25.0          | 72.8      | Sáng     |
| 21:45:27 26/11/2025 | 24.9          | 72.8      | Tối      |
| 21:45:17 26/11/2025 | 24.2          | 74.9      | Tối      |
| 21:45:07 26/11/2025 | 24.0          | 74.7      | Tối      |

## F. KẾT LUẬN & HƯỚNG MỞ RỘNG

### I. Kết luận

Trong khuôn khổ môn học IoT, nhóm chúng em đã xây dựng thành công một hệ thống nhà thông minh (Smart Home System) dựa trên nền tảng ESP32, giao thức MQTT và Web Dashboard điều khiển thời gian thực.

Hệ thống hoạt động theo mô hình khép kín: giám sát → xử lý → điều khiển → phản hồi, giúp tự động hóa nhiều chức năng trong gia đình và nâng cao mức độ tiện nghi – an toàn cho người dùng.

Về mặt kỹ thuật, dự án đã hoàn thiện đầy đủ các thành phần:

- Phần cứng

Các cảm biến như nhiệt độ – độ ẩm, ánh sáng, cùng với các thiết bị đầu ra như đèn, quạt, relay đều được kết nối và điều khiển ổn định, phản hồi nhanh.

#### ESP32 thực hiện:

- Đọc dữ liệu cảm biến liên tục
- Xử lý logic tự động (bật đèn khi trời tối, bật quạt khi nóng, ...)
- Gửi dữ liệu và nhận lệnh chuẩn xác qua MQTT
- Duy trì kết nối ổn định ngay cả khi nhiều thiết bị hoạt động cùng lúc.

## **Frontend**

Web Dashboard được thiết kế trực quan, cho phép:

- Theo dõi trạng thái nhà thông minh thời gian thực
- Điều khiển thiết bị từ xa (đèn, quạt, cảnh báo...)
- Xem thông số cảm biến

## **Backend**

Hệ thống tích hợp **MQTT Broker + API Server**, vận hành mượt mà, hỗ trợ:

- Nhiều thiết bị cùng lúc (multi-device)
- Lưu trữ dữ liệu
- Gửi lệnh điều khiển thời gian thực đến từng thiết bị.

## **II. Hướng mở rộng**

### **1. Bổ sung thêm cảm biến và thiết bị**

- Cảm biến khí CO/CO<sub>2</sub>
- Cảm biến mưa, cảm biến cửa từ, cảm biến chuyển động
- Hệ thống rèm cửa thông minh

### **2. Phát triển cơ chế cảnh báo từ xa**

- Gửi SMS, Telegram Bot

### **3. Nâng cấp bảo mật**

- Mã hóa MQTT bằng TLS (MQTTTS)
- Token/JWT Authentication
- Hệ thống đăng nhập người dùng nhiều quyền.

### **4. Tối ưu OTA và quản lý firmware**

- Kiểm tra phiên bản tự động
- Hỗ trợ rollback khi cập nhật lỗi
- Lưu log cập nhật theo thiết bị.

## **5. Hướng tới triển khai thực tế**

- Lắp đặt thực tế tại căn hộ hoặc văn phòng
- Sử dụng bộ nguồn ổn định và UPS dự phòng
- Thiết kế hộp bảo vệ, chống bụi, chống ẩm.

## G. Phân chia công việc và kế hoạch triển khai

### Phân chia công việc

| Thành viên        | Công việc                                                                    |
|-------------------|------------------------------------------------------------------------------|
| Bùi Tiến Dũng     | Thiết kế và lập trình vi điều khiển ESP32, tích hợp cảm biến                 |
| Nguyễn Duy Hiếu   | Xây dựng hệ thống Backend (API, database, MQTT/HTTP giao tiếp với ESP32)     |
| Đào Mạnh Đạt      | Phát triển giao diện người dùng (Web/App) để điều khiển và giám sát hệ thống |
| Nguyễn Quang Dũng | Thiết kế tài liệu, mô hình hệ thống, hỗ trợ tích hợp và kiểm thử tổng thể    |